



UNIVERSIDAD NACIONAL AUTÓNOMA DE
MÉXICO

FACULTAD DE INGENIERÍA
DIVISIÓN DE INGENIERÍA ELÉCTRICA
INGENIERÍA EN COMPUTACIÓN



LABORATORIO DE COMPUTACIÓN GRÁFICA e INTERACCIÓN HUMANO
COMPUTADORA

REPORTE DE PRÁCTICA N° 04

NOMBRE COMPLETO: Velazquez Caudillo Osbaldo

N° de Cuenta: 320341704

GRUPO DE LABORATORIO: 2

GRUPO DE TEORÍA: 6

SEMESTRE: 2026-1

FECHA DE ENTREGA LÍMITE: 21/09/25

CALIFICACIÓN: _____

1.- Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.

Ejercicio 1

1.- Terminar la Grúa con:

- cuerpo(prisma rectangular)
- base (pirámide cuadrangular)
- 4 llantas(4 cilindros) con teclado se pueden girar cada una de las 4 llantas por separado

Para llevar a cabo la práctica, primera creamos un cuerpo extra para trabajar con este como nodo padre el cual fue un cubo pero alargado, con base en este realizamos la jerarquía, pasamos a crear posteriormente la base con ayuda de una pirámide cuadrangular.

```
//cuerpo extra
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(1.5f, 4.5f, -4.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(7.5f, 4.0f, 1.0f));
model = glm::rotate(model, glm::radians(5.0f), glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(1.0f, 0.647f, 0.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[0] = RenderMesh();

//Base
model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, -2.5f, 0.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(0.5f, 2.5f, 2.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.7f, 0.7f, 0.7f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[4] = RenderMesh();

//LLANTAS DELANTERAS
//llanta-derecha
model = modelaux;
model = glm::translate(model, glm::vec3(-4.25f, -1.25f, 1.0f));
model = glm::scale(model, glm::vec3(2.0f, 2.0f, 1.0f));
model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
modelaux = model;
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion5()), glm::vec3(0.0f, 1.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.2f, 0.2f, 0.2f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[2] = RenderMeshGeometry();

//LLANTA-IZQUIERDA
model = modelaux;
model = glm::translate(model, glm::vec3(4.25f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion6()), glm::vec3(0.0f, 1.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.2f, 0.2f, 0.2f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[2] = RenderMeshGeometry();

//LLANTAS-TRASERAS
//LLANTA-DERECHA
model = modelaux;
model = glm::translate(model, glm::vec3(4.25f, -1.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion7()), glm::vec3(0.0f, 1.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.2f, 0.2f, 0.2f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[2] = RenderMeshGeometry();

//LLANTA-IZQUIERDA
model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, -1.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion8()), glm::vec3(0.0f, 1.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.2f, 0.2f, 0.2f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[2] = RenderMeshGeometry();

//cuerpo
```

Una vez creando estos dos cuerpos, pasamos a crear las llantas, cada una con una articulación para que su movimiento fuera independiente, ahora fue muy importante

hacer uso de la matriz auxiliar $modelaux$ para poder llevar a cabo la jerarquía, en este caso de la base con las llantas. Así de esta manera, logramos que por ejemplo las llantas se muevan de manera independiente, pero si llegara a existir un movimiento por parte de la base, entonces estas giraran juntas.

Las transformaciones de traslación, rotación y escala se aplican en un orden específico, donde los objetos hijos (como las articulaciones o llantas) dependen de las transformaciones aplicadas a sus padres (como la cabina o base). Esto significa que las jerarquías están siendo implementadas correctamente en el código

Ejecución



Ejercicio 2

2.- Crear un animal robot 3d

-Instanciando cubos, pirámides, cilindros, conos, esferas:

-4 patas articuladas en 2 partes (con teclado se puede mover las dos articulaciones de cada pata)

-cola articulada o 2 orejas articuladas. (con teclado se puede mover la cola o cada oreja independiente)

Para este ejercicio elegí la araña, para la cual me base en diferentes modelos



De cada uno trate de tomar una parte diferente, para tratar de hacerla lo mas realista posible, entonces inicialmente se creo la estructura del cuerpo de la araña para lo cual tome dos esferas e hice uso de un cilindro hecho con caras de 6 lados, esto para la parte del rostro de la araña, ya una vez hecha la parte del cuerpo, procedí a crear las patas, la ultima parte del cuerpo de la araña que es la esfera más grande se creó para simular una cola, y esta tiene un movimiento independiente tomando como referencia la esfera del cuerpo principal de la araña

Para las patas decidí usar cilindros muy delgados, de nueva cuenta tomando como referencia el cuerpo principal de la araña, además para crearlas se hizo en dos partes, entonces cada pata esta hecha por dos partes y conformadas por dos articulaciones en total 8 articulaciones con 8 partes, 2 para cada pata.

De igual manera opte por crearle orejas, que toman como nodo padre el rostro de la araña y tienen un movimiento independiente.

Para darle un toque más estético le cree ojos con pirámides y también quelíceros hechos con conos rotados.

En total se crearon 11 articulaciones, 8 para las patas 1 para la parte trasera (como cola) y 1 para cada oreja de la araña.

Cuerpo

```
etica4 (Ambito global)

meshList[2]->RenderMeshGeometry();

// ARAÑA
//Creando el cuerpo de la araña (esfera) como nodo padre**
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(0.0f, 4.0f, -4.0f));
modelBody = model; // Guardamos la matriz del cuerpo (para las patas y articulaciones)
model = glm::scale(model, glm::vec3(3.0f, 3.0f, 3.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.0f, 0.0f, 0.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
sp.render();

//Segunda parte del cuerpo de la araña (cilindro)
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(0.0f, 4.0f, 0.0f));
glm::mat4 modelBody2 = model;
model = glm::rotate(model, glm::radians(90.0f), glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::scale(model, glm::vec3(3.0f, 2.0f, 2.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.0f, 0.0f, 0.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
meshList[2]->RenderMeshGeometry();

//parte trasera (cola)
//Articulación para la parte trasera (conectada al cuerpo principal)
model = modelBody; // Usamos la matriz del cuerpo principal para la articulación
model = glm::translate(model, glm::vec3(0.0f, 0.0f, 3.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion9()), glm::vec3(0.0f, 0.0f, 1.0f));

model = glm::translate(model, glm::vec3(0.0f, 0.0f, -6.0f));
model = glm::scale(model, glm::vec3(4.0f, 4.0f, 4.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.0f, 0.0f, 0.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
sp.render();

model = glm::translate(model, glm::vec3(0.0f, 0.6f, 0.0f));
model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.8f, 0.6f, 0.6f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(1.0f, 0.0f, 0.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[2]->RenderMeshGeometry();
```

Ojos

```
// Ojo derecho
model = modelBody2; // Usamos la matriz del cuerpo 2 para los ojos
model = glm::translate(model, glm::vec3(1.0f, 0.5f, 2.5f));
model = glm::rotate(model, glm::radians(180.0f), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.3f, 0.3f, 0.5f));
color = glm::vec3(1.0f, 0.0f, 0.0f);
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[4]->RenderMeshGeometry(); // Asumimos que el modelo del ojo es el índice 4

// Ojo izquierdo
model = modelBody2; // Usamos la matriz del cuerpo 2 para el ojo izquierdo
model = glm::translate(model, glm::vec3(0.5f, 0.3f, 2.8f));
model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.7f));
color = glm::vec3(1.0f, 0.0f, 0.0f);
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[4]->RenderMeshGeometry();

// Ojo izquierdo
model = modelBody2; // Usamos la matriz del cuerpo 2 para el ojo izquierdo
model = glm::translate(model, glm::vec3(-1.0f, 0.5f, 2.2f));
model = glm::rotate(model, glm::radians(180.0f), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.3f, 0.3f, 0.5f));
color = glm::vec3(1.0f, 0.0f, 0.0f);
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[4]->RenderMeshGeometry();

// Ojo izquierdo
model = modelBody2; // Usamos la matriz del cuerpo 2 para el ojo izquierdo
model = glm::translate(model, glm::vec3(-0.4f, 0.3f, 2.8f));
model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.7f));
color = glm::vec3(1.0f, 0.0f, 0.0f);
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[4]->RenderMeshGeometry();
```

Quelíceros

```
Camera.cpp practica4_conjaraquia.cpp Window.cpp W
Practica4 (Ámbito global)

623 //Colmillos
624 model = modelBody2; // Usamos la matriz del cuerpo 2 como base
625 model = glm::translate(model, glm::vec3(0.5f, -0.5f, 2.4f));
626 model = glm::rotate(model, glm::radians(135.0f), glm::vec3(1.0f, 0.0f, 0.0f));
627 model = glm::scale(model, glm::vec3(0.1f, 1.0f, 0.1f));
628 color = glm::vec3(0.8f, 0.8f, 0.8f);
629 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
630 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
631 meshList[3] -> RenderMeshGeometry();
632
633 //Colmillo izquierdo (conectado al cuerpo 2)
634 model = modelBody2; // Usamos la matriz del cuerpo 2 para el colmillo izquierdo
635 model = glm::translate(model, glm::vec3(-0.5f, -0.5f, 2.2f));
636 model = glm::rotate(model, glm::radians(135.0f), glm::vec3(1.0f, 0.0f, 0.0f));
637 model = glm::scale(model, glm::vec3(0.1f, 1.0f, 0.1f));
638 color = glm::vec3(0.8f, 0.8f, 0.8f);
639 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
640 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
641 meshList[3] -> RenderMeshGeometry();
642
643
```

Orejas

```
// OREJA
// OREJA DERECHA
// Articulación
model = modelBody2;
model = glm::translate(model, glm::vec3(2.0f, 1.0f, 1.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion7()), glm::vec3(0.0f, 1.0f, 0.0f)); // Rotación de la articulación
modelOrejaD = model; // Guardamos la matriz de la articulación

model = modelOrejaD;
model = glm::translate(model, glm::vec3(0.0f, 0.5f, 0.0f));
model = glm::scale(model, glm::vec3(0.5f, 1.0f, 0.2f));
color = glm::vec3(0.0f, 0.0f, 0.0f);
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[3] -> RenderMeshGeometry();

// OREJA IZQUIERDA
// Articulación
model = modelBody2;
model = glm::translate(model, glm::vec3(-2.0f, 1.0f, 1.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion8()), glm::vec3(0.0f, 1.0f, 0.0f)); // Rotación de la articulación
modelOrejaI = model;

model = modelOrejaI;
model = glm::translate(model, glm::vec3(0.0f, 0.5f, 0.0f));
model = glm::scale(model, glm::vec3(0.5f, 1.0f, 0.2f));
color = glm::vec3(0.0f, 0.0f, 0.0f);
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[3] -> RenderMeshGeometry();
```

Patas

```
Camera.cpp practica4_conjaraquia.cpp Window.cpp Window.h
Practica4 (Ámbito global) main()

674 //PATAS DELANTERAS
675 // PATA DERECHA
676 // Primera parte de la pata
677 model = modelBody;
678 model = glm::translate(model, glm::vec3(2.5f, 0.0f, 1.0f)); // Posición de la articulación
679 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion1()), glm::vec3(1.0f, 0.0f, 0.0f)); // Rotación de la articulación 1
680
681 //Primera parte de la pata
682 model = glm::translate(model, glm::vec3(0.0f, 0.0f, 3.0f));
683 modelPata1 = model;
684 model = glm::rotate(model, glm::radians(180.0f), glm::vec3(0.0f, 0.0f, 1.0f));
685 model = glm::rotate(model, glm::radians(180.0f), glm::vec3(1.0f, 0.0f, 0.0f));
686 model = glm::scale(model, glm::vec3(0.2f, 0.0f, 0.2f));
687 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
688 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
689 meshList[2] -> RenderMeshGeometry();
690
691 //Segunda articulación
692 model = modelPata1;
693 model = glm::translate(model, glm::vec3(2.0f, 0.0f, 0.0f));
694 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion2()), glm::vec3(1.0f, 0.0f, 0.0f));
695
696 model = glm::translate(model, glm::vec3(-0.05f, -2.2f, 1.85f));
697 model = glm::rotate(model, glm::radians(25.0f), glm::vec3(0.0f, 0.0f, 1.0f));
698 model = glm::scale(model, glm::vec3(0.2f, 0.0f, 0.2f));
699 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
700 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
701 meshList[2] -> RenderMeshGeometry();
702
703 //PATA IZQUIERDA
704 model = modelBody;
705 model = glm::translate(model, glm::vec3(-2.5f, 0.0f, 1.0f));
706 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion3()), glm::vec3(1.0f, 0.0f, 0.0f));
707
708 model = glm::translate(model, glm::vec3(0.0f, 0.0f, 3.0f));
709 modelPata2 = model;
710 model = glm::rotate(model, glm::radians(-90.0f), glm::vec3(0.0f, 0.0f, 1.0f));
711 model = glm::rotate(model, glm::radians(180.0f), glm::vec3(1.0f, 0.0f, 0.0f));
712 model = glm::scale(model, glm::vec3(0.2f, 0.0f, 0.2f));
713 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
714 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
715 meshList[2] -> RenderMeshGeometry();
716
717 //Segunda articulación
718 model = modelPata2;
719 model = glm::translate(model, glm::vec3(2.0f, 0.0f, 0.0f));
720 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion4()), glm::vec3(1.0f, 0.0f, 0.0f));
721
```

```
camera.cpp  practica4_conjerarquia.cpp*  Window.cpp  Window.h
Practica4  (Ámbito global)
716 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
717 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
718 meshList[2]->RenderMeshGeometry();
719
720 //Segunda articulación
721 model = modelPata2;
722 model = glm::translate(model, glm::vec3(2.0f, 0.0f, 0.0f));
723 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion4()), glm::vec3(1.0f, 0.0f, 0.0f));
724
725 model = glm::translate(model, glm::vec3(-3.30f, -2.4f, 1.85f));
726 model = glm::rotate(model, glm::radians(-25.0f), glm::vec3(0.0f, 0.0f, 1.0f));
727 model = glm::scale(model, glm::vec3(0.2f, 5.0f, 0.2f));
728 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
729 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
730 meshList[2]->RenderMeshGeometry();
731
732 //patas traseras
733 //PATA DERECHA
734 model = modelBody;
735 model = glm::translate(model, glm::vec3(2.5f, 0.0f, -2.0f));
736 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion5()), glm::vec3(1.0f, 0.0f, 0.0f));
737
738 model = glm::translate(model, glm::vec3(2.1f, 0.0f, 2.0f));
739 modelPata3 = model;
740 model = glm::rotate(model, glm::radians(115.0f), glm::vec3(0.0f, 0.0f, 1.0f));
741 model = glm::scale(model, glm::vec3(0.2f, 6.2f, 0.2f));
742 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
743 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
744 meshList[2]->RenderMeshGeometry();
745
746 //Segunda articulación
747 model = modelPata3;
748 model = glm::translate(model, glm::vec3(2.0f, 0.0f, -2.0f));
749 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion6()), glm::vec3(1.0f, 0.0f, 0.0f));
750
751 model = glm::translate(model, glm::vec3(2.0f, -1.7f, 2.0f));
752 model = glm::rotate(model, glm::radians(25.0f), glm::vec3(0.0f, 0.0f, 1.0f));
753 model = glm::scale(model, glm::vec3(0.2f, 6.4f, 0.2f));
754 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
755 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
756 meshList[2]->RenderMeshGeometry();
757
758 //PATA IZQUIERDA
759
760 model = modelBody;
761 model = glm::translate(model, glm::vec3(-2.5f, 0.0f, -2.0f));
762 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion10()), glm::vec3(1.0f, 0.0f, 0.0f));
```

```
meshList[2]->RenderMeshGeometry();
//PATA IZQUIERDA
model = modelBody;
model = glm::translate(model, glm::vec3(-2.5f, 0.0f, -2.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion10()), glm::vec3(1.0f, 0.0f, 0.0f));
//primera parte de la pata
model = glm::translate(model, glm::vec3(-2.1f, 0.0f, 2.0f));
modelPata4 = model;
model = glm::rotate(model, glm::radians(-115.0f), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::scale(model, glm::vec3(0.2f, 6.1f, 0.2f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[2]->RenderMeshGeometry();
//Segunda articulación (conectada a la primera parte de la pata)
model = modelPata4;
model = glm::translate(model, glm::vec3(2.0f, 0.0f, -2.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion11()), glm::vec3(1.0f, 0.0f, 0.0f));
color = glm::vec3(0.0f, 0.0f, 0.0f);
model = glm::translate(model, glm::vec3(-6.0f, -1.7f, 2.0f));
model = glm::rotate(model, glm::radians(-25.0f), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::scale(model, glm::vec3(0.2f, 6.3f, 0.2f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[2]->RenderMeshGeometry();
```

Importante mencionar que cada una tiene un movimiento independiente de esta manera tenemos la siguiente configuración de teclas para el movimiento de cada una de las articulaciones.

TECLAS

PATAS DELANTERAS

Pata derecha

F-> Mueve la primera parte de la pata de la araña (articulación pegada al cuerpo)

G-> Mueve la segunda parte de la pata de la araña

Pata izquierda

H-> mueve la primera parte de la pata de la araña (articulación pegada al cuerpo)

J-> Mueve la segunda parte de la pata de la araña

PATAS TRASERAS

Pata derecha

K-> mueve la primera parte de la pata de la araña (articulación pegada al cuerpo)

L-> Mueve la segunda parte de la pata de la araña

Pata izquierda

O-> mueve la primera parte de la pata de la araña (articulación pegada al cuerpo)

P-> Mueve la segunda parte de la pata de la araña

T -> Parte trasera de la araña

N-> Oreja izquierda de la araña

M-> Oreja derecha de la araña

Declaración de matrices auxiliares

```
325 sp.load();//enviar la esfera al shader
326
327 glm::mat4 model(1.0);//Inicializar matriz de Modelo 4x4
328 glm::mat4 modelaux(1.0);//Inicializar matriz de Modelo 4x4
329 //nuevas matrices
330 glm::mat4 modelBody(1.0);
331 glm::mat4 modelBody2(1.0);
332 glm::mat4 modelOrejaD(1.0);
333 glm::mat4 modelOrejaI(1.0);
334 glm::mat4 modelC(1.0);
335 glm::mat4 modelPata1(1.0);
336 glm::mat4 modelPata2(1.0);
337 glm::mat4 modelPata3(1.0);
338 glm::mat4 modelPata4(1.0);
339 glm::vec3 color = glm::vec3(0.0f, 0.0f, 0.0f); //inicializar Color para enviar a variable Uniform;
340
341 while (!mainWindow.getShouldClose())
```

Window.cpp

Teclas para mover articulaciones

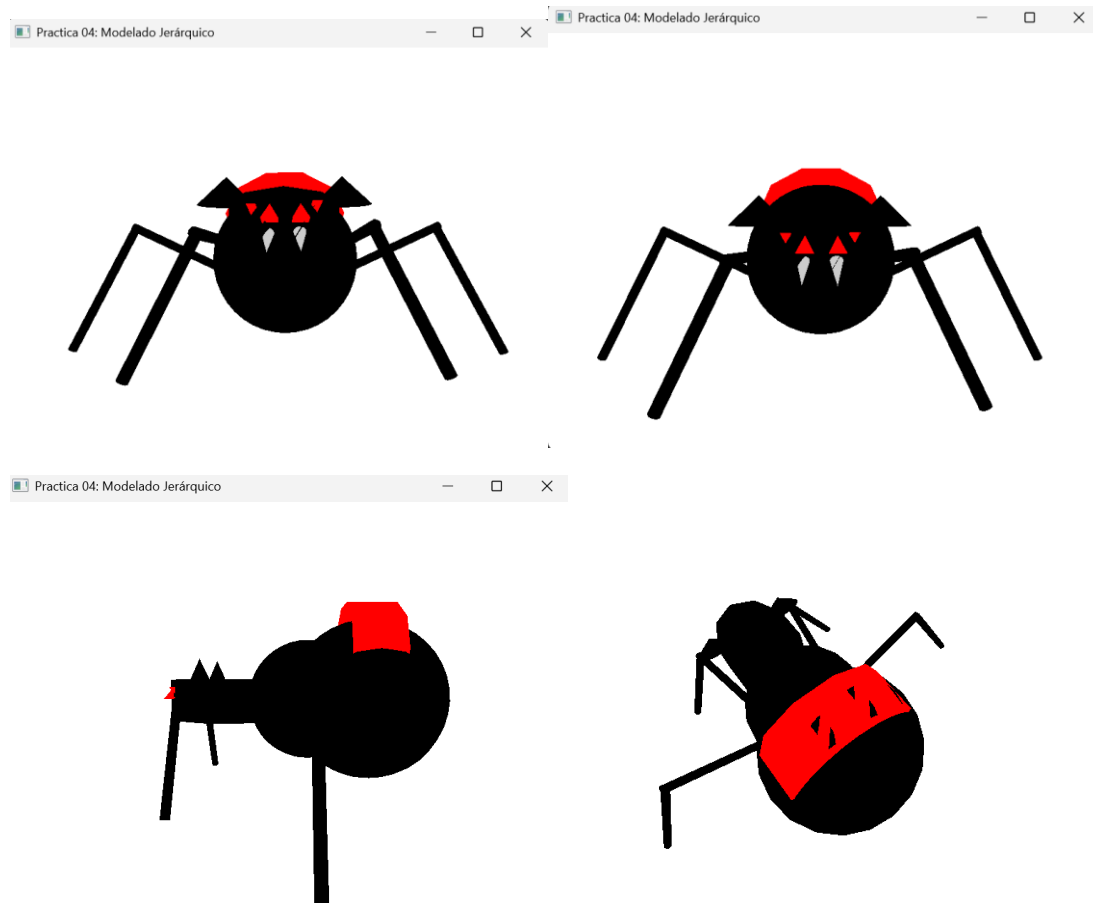
```
Camera.cpp  practica4_conjearquia.cpp  Window.cpp  Window.h
Practica4  Window
154  if (key == GLFW_KEY_L)
155  {
156      theWindow->articulacion6 += 10.0;
157  }
158  if (key == GLFW_KEY_M)
159  {
160      theWindow->articulacion7 += 10.0;
161  }
162
163  if (key == GLFW_KEY_N)
164  {
165      theWindow->articulacion8 += 10.0;
166  }
167  if (key == GLFW_KEY_T)
168  {
169      theWindow->articulacion9 += 10.0;
170  }
171  if (key == GLFW_KEY_O)
172  {
173      theWindow->articulacion10 += 10.0;
174  }
175  if (key == GLFW_KEY_P)
176  {
177      theWindow->articulacion11 += 10.0;
178  }
179
180  if (key == GLFW_KEY_D && action == GLFW_PRESS)
```

Window.h

```
Practica4  Window
10  Window(GLint windowWidth, GLint windowHeight);
11  int Initialise();
12  GLfloat getBufferWidth() { return bufferWidth; }
13  GLfloat getBufferHeight() { return bufferHeight; }
14  bool getShouldClose() {
15      return glfwWindowShouldClose(mainWindow);
16  }
17  bool* getKeys() { return keys; }
18  GLfloat getChangeX();
19  GLfloat getChangeY();
20  void swapBuffers() { return glfwSwapBuffers(mainWindow); }
21  GLfloat getrotax() { return rotax; }
22  GLfloat getrotay() { return rotay; }
23  GLfloat getrotaz() { return rotaz; }
24  GLfloat getarticulacion1() { return articulacion1; }
25  GLfloat getarticulacion2() { return articulacion2; }
26  GLfloat getarticulacion3() { return articulacion3; }
27  GLfloat getarticulacion4() { return articulacion4; }
28  GLfloat getarticulacion5() { return articulacion5; }
29  GLfloat getarticulacion6() { return articulacion6; }
30  GLfloat getarticulacion7() { return articulacion7; }
31  GLfloat getarticulacion8() { return articulacion8; }
32  GLfloat getarticulacion9() { return articulacion9; }
33  GLfloat getarticulacion10() { return articulacion10; }
34  GLfloat getarticulacion11() { return articulacion11; }
35
36
37
38  ~Window();
39
40  private:
41  GLFWwindow *mainWindow;
42  GLint width, height;
43  GLfloat rotax, rotay, rotaz, articulacion1, articulacion2, articulacion3, articulacion4, articulacion5, articulacion6, articulacion7, articulacion8, articulacion9, articulacion10, articulacion11;
44  bool keys[1024];
45  GLint bufferWidth, bufferHeight;
46  GLfloat lastX;
47  GLfloat lastY;
48  GLfloat xChange;
49  GLfloat yChange;
50  bool mouseFirstMoved;
51  void createCallback();
52  static void ManejaTeclado(GLFWwindow* window, int key, int code, int action, int mode);
53  static void ManejaMouse(GLFWwindow* window, double xPos, double yPos);
54
55  ;
```

Se hizo cambio de color en el fondo para que se lograran apreciar los colores, además se modificó el archivo Camera.cpp para tener un mejor control sobre la cámara y que se puede subir presionando la tecla de espacio y para bajar con la tecla Ctrl.

Ejecución



2.- Liste los problemas que tuvo a la hora de hacer estos ejercicios y si los resolvió explicar cómo fue, en caso de error adjuntar captura de pantalla

No se presentaron errores

Conclusión

Esta práctica me resultó complicada al principio, principalmente porque no comprendía completamente el concepto de jerarquía y todavía me cuesta un poco manejarlo. Sin embargo, me permitió darme cuenta de la importancia de utilizar estructuras jerárquicas, especialmente al trabajar con objetos complejos. Aunque fue una práctica difícil, me agrado más que la anterior; el video que se compartió en Classroom fue de gran ayuda para entender cómo funcionan las jerarquías. Considero que este ejercicio es bueno para aprender a manipular figuras y cuerpos geométricos y, sobre todo, para comprender mejor el espacio de trabajo en 3D.

Bibliografía

1. Araña Modelo 3D gratis - .blend .fbx - Free3D. (n.d.).
<https://free3d.com/es/modelo-3d/spider-257546.html>
2. modelo 3d Araña Low Poly - TurboSquid 1727159. (2021, May 1).
TurboSquid. <https://www.turbosquid.com/es/3d-models/3d-low-poly-spider-1727159>